

Co-funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Education and Culture Executive Agency (EACEA). Neither the European Union nor EACEA can be held responsible for them.

2023-1-EL01-KA220-HED-000166184

Final Project, Option 1: Where does your CNN make its mistakes?

Software that reads handwritten numbers, such as postcodes on envelopes or account numbers on paper forms, must be judged not only on how often it is right but on where it goes wrong, because a confident misreading carries a real cost. In this project you build such a model and study the mistakes it makes.

In this project you build a convolutional neural network that recognises handwritten digits, then you study the mistakes it makes.

Central question: where does your CNN make its mistakes, and does one change to the model move them?

The data set is MNIST: 70,000 images of handwritten digits, each 28 by 28 pixels in greyscale, labelled 0 to 9. You download it yourself in the next cell.

This project draws on two earlier modules. In Module 5, we built and trained dense neural networks in Keras. In Module 6, we added convolutional layers, used Dropout and EarlyStopping to control overfitting, and read a confusion matrix to see which classes a model mixes up. Here you bring those skills together on a larger, more varied data set.

How this notebook is organised. The cells in Part A are already written for you. They handle the import, the download, and the reshaping that turns each flat row of 784 numbers back into a 28 by 28 image. From Part B onward, the cells are yours to complete. Each one carries comments describing what to do, along with a few hints, but not the finished code. Work through them in order.

Training on the full data set may take a few minutes per epoch on a laptop without a graphics card. This is expected.

Part A. Setup and data

We start with the imports. This cell loads everything the notebook needs, so we run it once at the top.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import keras
from keras import layers
```

Loading the data

We download MNIST from OpenML. The call returns the images in `X` as 70,000 rows of 784 numbers, and the labels in `y` as text. We convert the labels to integers so the model can work with them. The download runs once and may take a moment.

```
X, y = fetch_openml("mnist_784", version=1, as_frame=False, return_X_y=True)
y = y.astype(int)

print("Images:", X.shape)
print("Labels:", y.shape, "| classes:", np.unique(y))
```

```
Images: (70000, 784)
Labels: (70000,) | classes: [0 1 2 3 4 5 6 7 8 9]
```

Preparing the images

Each row of `X` is one image flattened into a line of 784 numbers. We reshape every row back into a 28 by 28 image with a single greyscale channel, and we scale the pixel values from the range 0 to 255 down to the range 0 to 1, which helps the network train. We then split the data into a training set and a test set, keeping the class balance with `stratify`, as we did in Module 3.

```
X = X.reshape(-1, 28, 28, 1).astype("float32") / 255.0

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("Training images:", X_train.shape)
print("Test images:    ", X_test.shape)

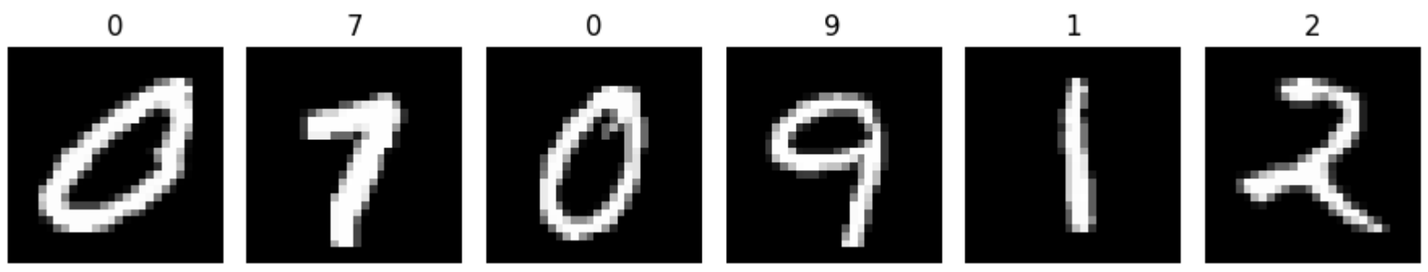
# If training is slow on your machine, you may work with a smaller sample
# while you get everything running, for example:
#   X_train, y_train = X_train[:10000], y_train[:10000]
```

```
Training images: (56000, 28, 28, 1)
Test images:     (14000, 28, 28, 1)
```

A first look at the images

Before modelling, we look at a few training images with their labels, to confirm the data loaded correctly.

```
fig, axes = plt.subplots(1, 6, figsize=(9, 2))
for ax, image, label in zip(axes, X_train, y_train):
    ax.imshow(image.reshape(28, 28), cmap="gray")
    ax.set_title(str(label))
    ax.axis("off")
plt.tight_layout()
plt.show()
```



Part B. Build and train your network

From here on, the cells are yours to complete.

First, build a convolutional neural network with the Keras Sequential API, the same style you used in Module 6. Recall the shape of one image: 28 by 28, with one channel.

```
# Build a small convolutional network with the Keras Sequential API.
#
# A sensible starting point has these layers, in order:
#   - an Input layer with the image shape (28, 28, 1)          [given below]
#   - a Conv2D layer      (try 32 filters, a 3 by 3 window, relu activation)
#   - a MaxPooling2D layer (a 2 by 2 window)
#   - a Dropout layer      (0.3 is a reasonable value to start with)
#   - a Flatten layer      (moves from the 2D feature maps to a 1D vector)
#   - a Dense hidden layer (try 64 units, relu)
#   - a Dense output layer (one unit per class, softmax activation)
#
# TODO: complete the layer list below.
# Fix the random seed so that rerunning the notebook gives comparable results.
keras.utils.set_random_seed(42)

model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
    # TODO: add the Conv2D and MaxPooling2D layers
    # TODO: add Dropout
    # TODO: add Flatten, then the two Dense layers

model.summary()
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791705546.341894      49 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13756 MB memory:  -> device: 0, name: Tesla T4, pci bus i
I0000 00:00:1791705546.346003      49 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:1 with 13756 MB memory:  -> device: 1, name: Tesla T4, pci bus i
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
dropout (Dropout)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 64)	346,176
dense_1 (Dense)	(None, 10)	650

```
Total params: 347,146 (1.32 MB)
Trainable params: 347,146 (1.32 MB)
```

Compile the model

Compiling sets the optimiser, the loss function, and the metric to watch during training.

```
# Compile the model. You need to choose:
#   - an optimiser: "adam" is a solid default
#   - a loss function: the labels are integers 0 to 9, not one-hot vectors,
#     so use the crossentropy loss that works with integer labels
#     (the same one you used in Module 6)
#   - a metric to watch: "accuracy"
#
# TODO: call model.compile(...) with those three arguments.

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

Train the model

Train the network, watching how it performs on data it has not learnt from. `EarlyStopping` halts training once the validation loss stops improving, so you do not waste time or overfit.

```
# Train the model. Use:
#   - validation_split=0.1 so 10% of the training data is held back for checking
#   - an EarlyStopping callback: monitor "val_loss", patience of about 3,
#     and restore_best_weights=True
#   - up to about 15 epochs, and batch_size=128
#
# TODO: create the EarlyStopping callback.
# TODO: call model.fit(...) and store the returned object in `history`.
from keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=15,
    batch_size=128,
    callbacks=[early_stopping]
)
```

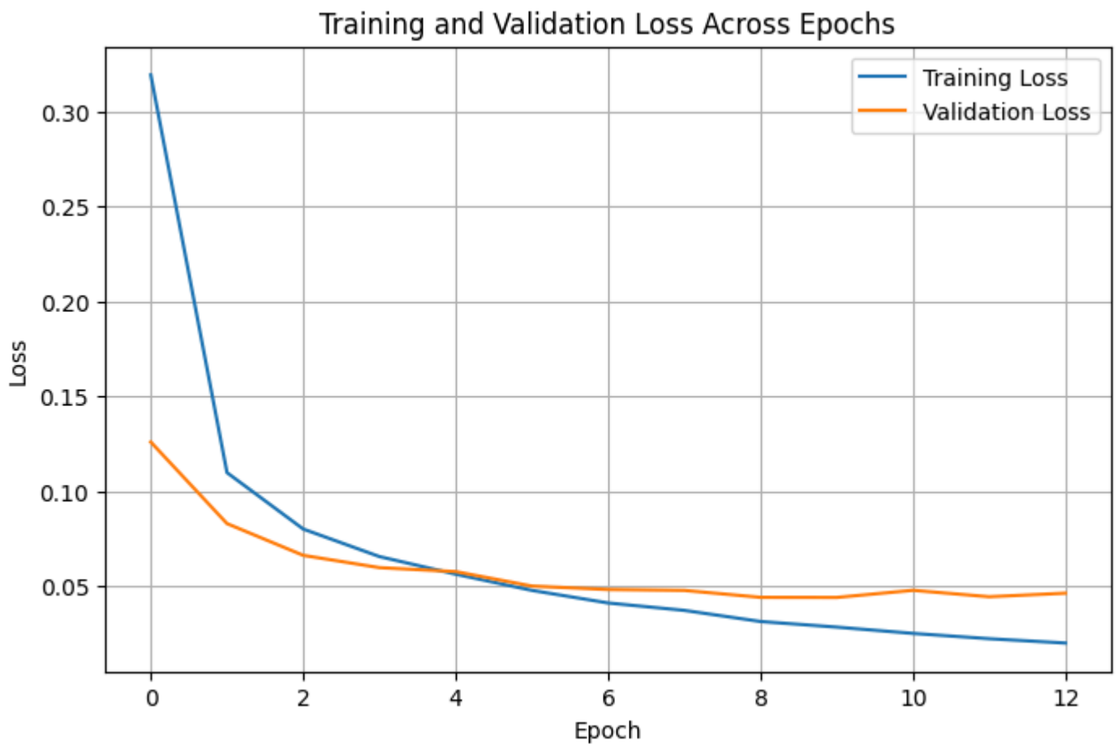
```
Epoch 1/15
 51/394 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.5597 - loss: 1.5251I0000 00:00:1791705578.409232     148 device_compiler.h:196] Compiled cluster using XLA!  This line is log
394/394 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9101 - loss: 0.3194 - val_accuracy: 0.9630 - val_loss: 0.1258
Epoch 2/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9678 - loss: 0.1097 - val_accuracy: 0.9757 - val_loss: 0.0828
Epoch 3/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9762 - loss: 0.0799 - val_accuracy: 0.9787 - val_loss: 0.0660
Epoch 4/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9797 - loss: 0.0654 - val_accuracy: 0.9802 - val_loss: 0.0595
Epoch 5/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9825 - loss: 0.0561 - val_accuracy: 0.9818 - val_loss: 0.0575
Epoch 6/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9853 - loss: 0.0476 - val_accuracy: 0.9864 - val_loss: 0.0498
Epoch 7/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9872 - loss: 0.0409 - val_accuracy: 0.9864 - val_loss: 0.0480
Epoch 8/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9884 - loss: 0.0370 - val_accuracy: 0.9859 - val_loss: 0.0475
Epoch 9/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9905 - loss: 0.0311 - val_accuracy: 0.9882 - val_loss: 0.0439
Epoch 10/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9908 - loss: 0.0282 - val_accuracy: 0.9871 - val_loss: 0.0438
Epoch 11/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9921 - loss: 0.0249 - val_accuracy: 0.9891 - val_loss: 0.0476
Epoch 12/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9927 - loss: 0.0221 - val_accuracy: 0.9871 - val_loss: 0.0442
Epoch 13/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9936 - loss: 0.0198 - val_accuracy: 0.9877 - val_loss: 0.0461
```

Learning curves

Plot the training and validation loss across the epochs. A validation curve that flattens or turns upward while the training curve keeps falling is a sign of overfitting.

```
# Plot the training and validation loss from history.history.
# The keys you need are "loss" and "val_loss".
#
# TODO: plot both curves on the same axes, add a legend and axis labels.

plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss Across Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Part C. Read the mistakes

Now measure how well the model does on the test set, the images it has never seen. This gives a reliable estimate of how it will perform on new images.

```
# Evaluate the trained model on the test set, and produce predictions.
#   - model.evaluate(X_test, y_test) returns the loss and the accuracy; store
#     them as test_loss and test_accuracy so you can compare the models later
#   - model.predict(X_test) returns a probability for each of the 10 classes;
#     the predicted digit is the class with the highest probability,
#     which you can get with .argmax(axis=1)
```

```
#
# TODO: create test_loss and test_accuracy, then print the test accuracy.
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy:.4f}")

# TODO: build y_pred, the predicted class for each test image.
y_probs = model.predict(X_test)
y_pred = y_probs.argmax(axis=1)
```

```
438/438 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9856 - loss: 0.0468
Test loss: 0.0468
Test accuracy: 0.9856
438/438 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

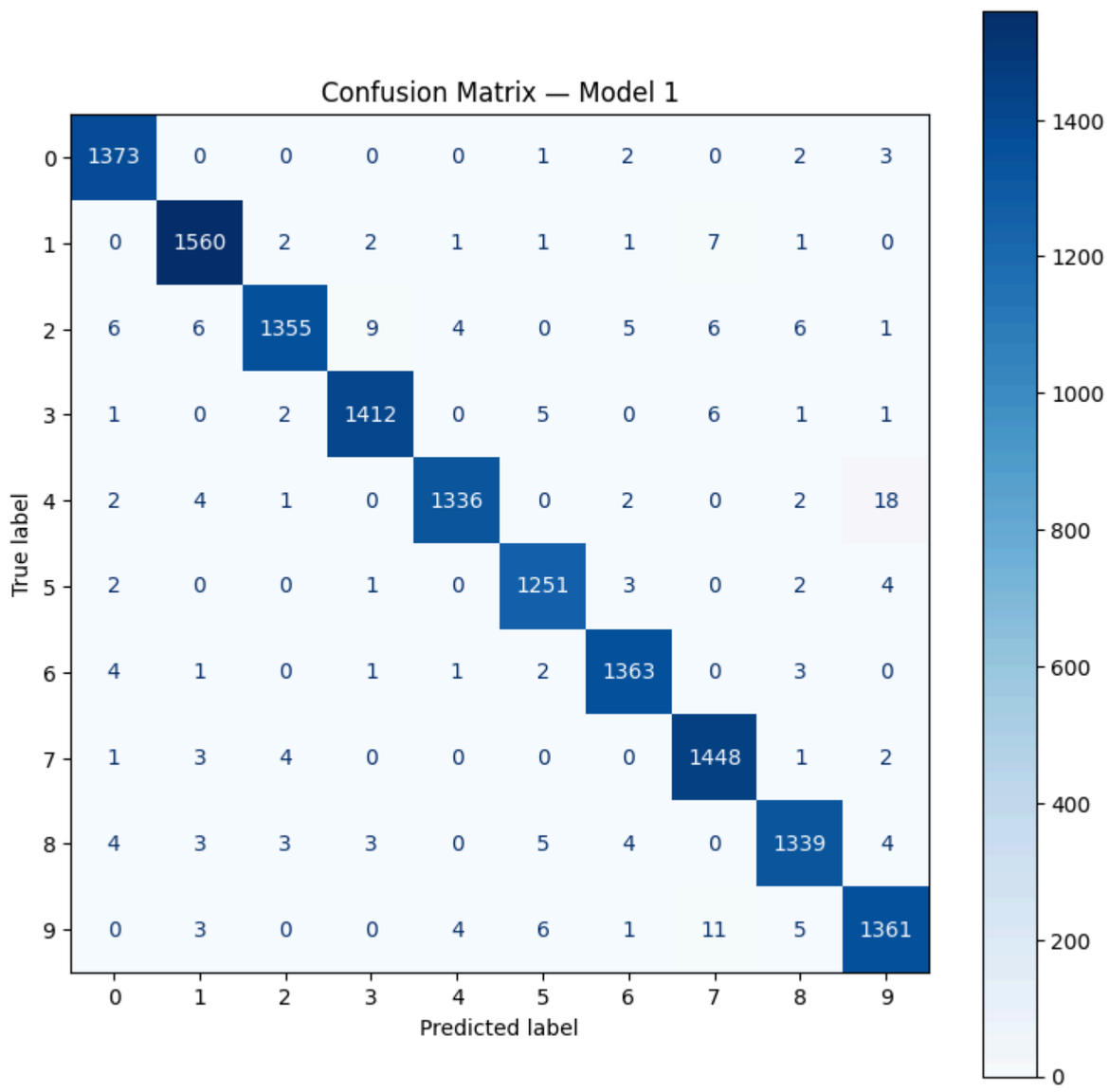
The confusion matrix

The confusion matrix is where you answer the first half of the central question: where the mistakes are. Read it by rows. Each row is a true digit, each column is the digit the model predicted. The diagonal holds the correct predictions; every off-diagonal cell is a mistake, and its position tells you which digit was mistaken for which.

```
# Build and display the confusion matrix comparing y_test with y_pred.
# Use confusion_matrix(...) to compute it and ConfusionMatrixDisplay(...) to draw it.
#
# TODO: compute the confusion matrix and display it.

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)

fig, ax = plt.subplots(figsize=(9, 9))
disp.plot(ax=ax, cmap="Blues", values_format="d")
plt.title("Confusion Matrix — Model 1")
plt.show()
```



What did you find?

Look at the off-diagonal cells with the largest counts. Which pairs of digits does your model confuse most often? Some pairs are visually similar and are commonly mixed up.

Write two or three sentences here naming the two most common confusion pairs, recording the count for each one, and suggesting why those particular digits might be hard to tell apart. For example, if row 4 and column 9 contains 18, the model predicted 9 for 18 images that were actually 4.

Examining the off-diagonal cells of the confusion matrix, the most common mistakes arise from the cells with the largest counts. Specifically, the model seems to misinterpret 4s as 9s in 18 instances, and 9s as 7s in 11 instances. This confusion is expected, since handwritten 4 and 9 are very similar digits, with the same structure, only the lines in one of them are drawn in angles and in the other one are curvy. Respectively, 9 and 7 shapes are also similar, with just one extra curvy line at the top in number 9. Overall, these digits are visually similar.

Part D. One change

Now answer the second half of the central question: does one change to the model move the mistakes?

Choose **one** change from the two below. Do not make both at once, or you will not know which one caused any difference you see.

- **Option A: a deeper network.** Before the Flatten layer, add `Conv2D(64, (3, 3), activation="relu")` followed by `MaxPooling2D((2, 2))`. This lets the network learn more detailed patterns.
- **Option B: stronger regularisation.** Raise the Dropout rate from 0.3 to 0.5. This makes the network rely less on any single feature.

Before you run anything, write down your prediction: will your chosen change raise the test accuracy, and will it reduce the confusion between the pairs you found in Part C? Note it here.

Neural-network training contains some randomness. A very small difference in accuracy does not necessarily show that one model is genuinely better. Look at the size of the difference and at the specific mistakes, not only at which accuracy is larger.

My prediction is that by changing the model into a deeper network, it will perform better, with higher accuracy and less misinterpretations in the confusion matrix. Adding second convolutional and max pooling layers will enable better and deeper understanding of the handwritten digits. The deeper architecture will be able to capture more complex spatial features, enabling the interpretation of curved lines rather than basic edges. This would provide the model with more capacity to distinguish the visually similar shapes.

```
# Build a second model that applies your ONE chosen change.
# Start from your Part B model and change only what Option A or Option B asks for.
#
# Use the same seed as before to make the comparison more reproducible.
keras.utils.set_random_seed(42)

# TODO: define model2 with your single change, then call model2.summary().
model2 = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    # change
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_1 (Dropout)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 64)	102,464
dense_3 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)

```
# Compile and train model2 exactly as you trained the first model,
# so the comparison is fair (same optimiser, loss, metric, callback, and settings).
#
# TODO: compile model2, then fit it and store the result in history2.

model2.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

from keras.callbacks import EarlyStopping

early_stopping2 = EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history2 = model2.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=15,
    batch_size=128,
    callbacks=[early_stopping2]
)
```

```
Epoch 1/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 8s 12ms/step - accuracy: 0.9197 - loss: 0.2730 - val_accuracy: 0.9775 - val_loss: 0.0769
Epoch 2/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9768 - loss: 0.0759 - val_accuracy: 0.9796 - val_loss: 0.0593
Epoch 3/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9832 - loss: 0.0551 - val_accuracy: 0.9857 - val_loss: 0.0477
Epoch 4/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9859 - loss: 0.0447 - val_accuracy: 0.9852 - val_loss: 0.0487
Epoch 5/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9884 - loss: 0.0381 - val_accuracy: 0.9873 - val_loss: 0.0429
Epoch 6/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9897 - loss: 0.0320 - val_accuracy: 0.9884 - val_loss: 0.0374
Epoch 7/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9911 - loss: 0.0269 - val_accuracy: 0.9854 - val_loss: 0.0440
Epoch 8/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9917 - loss: 0.0249 - val_accuracy: 0.9886 - val_loss: 0.0413
Epoch 9/15
394/394 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9930 - loss: 0.0214 - val_accuracy: 0.9882 - val_loss: 0.0375
```

```
# Compare the two models.
#   - store the results as test_loss2 and test_accuracy2
#   - print test_accuracy2 next to the first model's test_accuracy
#   - build y_pred2 and draw the new confusion matrix
#   - compare the counts for the two confusion pairs you identified in Part C
#
# TODO: evaluate model2, then compute and display its confusion matrix.

# Evaluate the second model
test_loss2, test_accuracy2 = model2.evaluate(X_test, y_test)

# Print both accuracies side-by-side for comparison
print(f"Model 1 Test accuracy: {test_accuracy:.4f}")
print(f"Model 2 Test accuracy: {test_accuracy2:.4f}")
print("-" * 30)

# Generate predictions for the second model
y_probs2 = model2.predict(X_test)
```

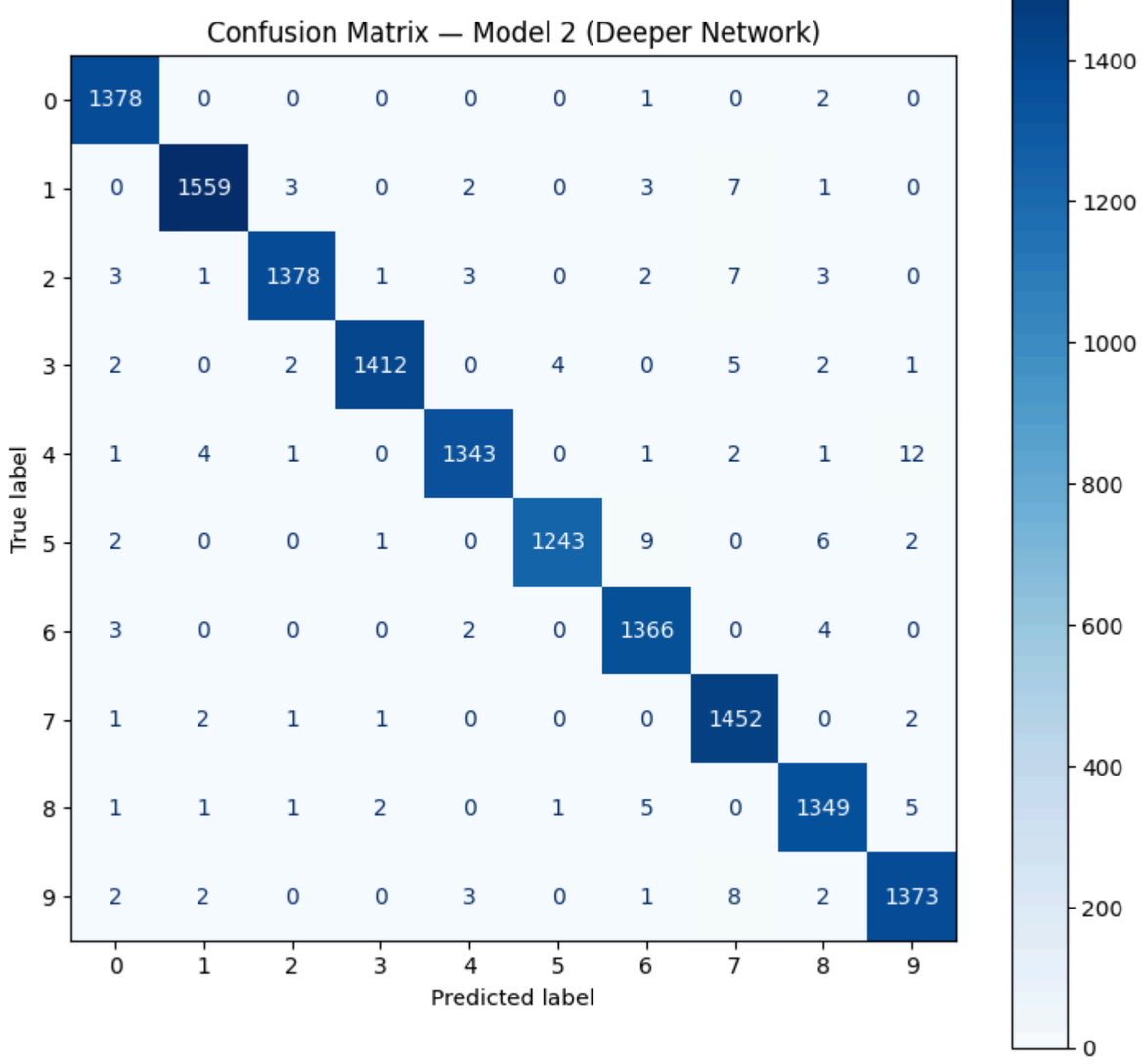
```
y_pred2 = y_probs2.argmax(axis=1)

# Compare the specific confusion pairs from Part C (True 4 -> Pred 9, True 9 -> Pred 7)
print("Specific Confusion Pairs Comparison:")
print(f"True 4 predicted as 9 --> Model 1: {cm[4, 9]} mistakes | Model 2: {confusion_matrix(y_test, y_pred2)[4, 9]} mistakes")
print(f"True 9 predicted as 7 --> Model 1: {cm[9, 7]} mistakes | Model 2: {confusion_matrix(y_test, y_pred2)[9, 7]} mistakes")

# Compute and display the new confusion matrix
cm2 = confusion_matrix(y_test, y_pred2)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2)

fig, ax = plt.subplots(figsize=(9, 9))
disp2.plot(ax=ax, cmap="Blues", values_format="d")
plt.title("Confusion Matrix — Model 2 (Deeper Network)")
plt.show()
```

```
438/438 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9895 - loss: 0.0348
Model 1 Test accuracy: 0.9856
Model 2 Test accuracy: 0.9895
------------------------------
438/438 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Specific Confusion Pairs Comparison:
True 4 predicted as 9 --> Model 1: 18 mistakes | Model 2: 12 mistakes
True 9 predicted as 7 --> Model 1: 11 mistakes | Model 2: 8 mistakes
```



Ethical considerations

A digit reader does not perform equally well for everyone. Handwriting varies by age, country, and schooling, and a model trained mostly on one style may read another less reliably. **Write two or three sentences here** on the risks of putting this model into a real reading pipeline. Consider whose handwriting might be under-represented in the training data, and what the consequence of a confident but wrong reading would be in the setting above.

In a real-world pipeline, the model risks misinterpreting the handwriting of particular individuals, for example foreigners, people of older or younger age, and people with varying educational backgrounds, since it is trained on specific writing styles. Such cases might be under-represented in the training data, so the model would not have the chance to identify the corresponding patterns. In high-stakes settings, like postal sorting center or bank transactions, a confident but incorrect prediction could misdirect important documents to the false address. Overall, this would result in discrimination against certain demographics and place a burden, demanding constant correction of these errors.

Wrapping up

Bring your findings together. **Write a short paragraph** that answers the central question directly:

- Where did your first model make most of its mistakes?
- Did your prediction in Part D turn out to be correct?
- For each of the two confusion pairs you identified, what was the count before and after the change?
- Did your one change move the mistakes, and is the difference large enough to be meaningful?

What to hand in: this completed notebook, with both confusion matrices visible in the output, and your written answers in the three markdown cells above (Part C, Part D, and this one).

The first model made the most of its mistakes on the identification of visually similar digits, specifically misinterpreting 4s for 9s and 9s for 7s. The prediction that an introduction of a deeper model would result in better accuracy and correct interpretation, by learning more complex spatial relations, turned out to be correct, with overall test accuracy elevating from 98.59% to 98.95%. In particular, based on the confusion matrices, the first model misidentified 4s for 9s in 18 instances, reduced to 12 instances in the second model. Additionally, regarding the 9s to 7s misinterpretation, the mistake was reduced from 11 instances in the first model to 8 in the second one. Overall, this single change in the model's architecture proved successful in the error reduction, while this difference appears to be large enough to be meaningful. The second model produced less errors than the first one, indicating that the deeper architecture successfully captured and learned more complex spatial features required for interpreting the visually similar handwritten shapes.